

Tema 5: Gestión de Estado Global con Redux

A medida que una aplicación React crece en complejidad, gestionar el estado exclusivamente mediante `useState` a nivel local resulta insostenible. Para solucionar este problema arquitectónico, integramos Redux.

1. Motivación: El problema del *Prop Drilling*

En una arquitectura puramente basada en React, si un componente anidado profundamente necesita acceder a un estado definido en el componente raíz (por ejemplo, los datos del usuario autenticado), dicho estado debe pasarse a través de las propiedades (`props`) de todos los componentes intermedios, incluso si estos no lo utilizan.

- **Consecuencias negativas:** Esto genera un alto acoplamiento, dificulta el mantenimiento del código y provoca renderizaciones innecesarias en el Virtual DOM de los componentes intermedios.
 - **La solución (Redux):** Extraer el estado global de los componentes y centralizarlo en un almacén externo (el *Store*). Cualquier componente puede "conectarse" directamente al Store para leer datos o despachar acciones, independientemente de su posición en el árbol de componentes.
-

2. Introducción a Redux (Conceptos Core)

Redux se fundamenta en tres pilares estrictos. Comprender su flujo unidireccional es vital para el examen.

2.1. El *Store* (La Fuente Única de Verdad)

Toda la información global de la aplicación reside en un único objeto centralizado. En nuestro proyecto, el Store se configura en `frontend/src/store/index.js` combinando los reducers de los distintos módulos.

2.2. Las Acciones (*Actions*)

El estado en Redux es de solo lectura. La única forma de modificarlo es emitiendo una acción.

- Una acción es un objeto JavaScript plano que describe *qué* ha sucedido.

- **Regla estricta:** Toda acción debe tener obligatoriamente una propiedad `type`.

```
// Ejemplo de Acción
{
  type: 'LOGIN_COMPLETED',
  authenticatedUser: { id: 1, userName: 'admin', role: 'ADMIN' }
}
```

2.3. Los Reductores (*Reducers*)

Son funciones puras encargadas de especificar cómo cambia el estado de la aplicación en respuesta a una acción.

- **Firma de la función:** `(estadoAnterior, accion) => nuevoEstado`.
- **Regla de Examen (Inmutabilidad):** Un reductor **jamás** debe mutar el `estadoAnterior`. Debe crear y devolver un objeto de estado completamente nuevo si hay cambios. Si la acción no le incumbe, debe devolver el `estadoAnterior` intacto.

2.4. Combinación de Reductores (`combineReducers`)

Para no tener una única función reductora gigantesca, Redux proporciona `combineReducers`. Esto nos permite dividir el estado global en módulos (por ejemplo: `users`, `catalog`, `shopping`) y asignar un reductor específico a cada uno.

```
// Ejemplo en nuestro proyecto
const rootReducer = combineReducers({
  catalog: catalog.reducer,
  shopping: shopping.reducer,
  users: users.reducer
});
```

3. Estado Normalizado

Almacenar datos en Redux requiere planificación. El **Estado Normalizado** es un patrón de diseño que consiste en estructurar el estado global de forma similar a una base de datos relacional.

- **El problema:** Si almacenamos objetos profundamente anidados (ej. un `Pedido` que contiene un array de `LineasDePedido`, que a su vez contienen objetos `Producto`), actualizar la información de un producto específico obligaría a clonar y reescribir todo el árbol de objetos debido a las reglas de inmutabilidad.

- **La solución normalizada:** Mantener el estado lo más plano posible. Las entidades se almacenan en diccionarios (objetos indexados por su ID) y las relaciones se modelan almacenando únicamente los IDs (como claves foráneas), no los objetos completos.
-

4. Criterios de Evaluación Frecuentes (Casos de Examen)

- **Estado Local vs Estado Global:** No todo debe ir a Redux. El estado que pertenece exclusivamente a la interfaz de usuario (por ejemplo, el texto temporal que un usuario está escribiendo en un formulario, o los errores de validación al enviarlo) debe mantenerse como estado local del componente usando `useState`. Almacenar errores de formularios locales en Redux es considerado una **mala práctica evaluable**.
- **Operaciones Asíncronas (Thunks):** Los reductores no pueden ejecutar operaciones asíncronas (como hacer un `fetch` a nuestro backend Spring). Para ello, las peticiones HTTP se realizan en funciones llamadas *Action Creators* (usando el middleware Redux Thunk). Solo después de que el backend responda con éxito, el Action Creator despacha la acción síncrona (ej. `LOGIN_COMPLETED`) para que el reductor actualice el Store.